



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algoritmusok és Adatszerkezetek I.

Pusztai Gábor

Algoritmusok és Alkalmazásaik Tanszék
Informatikai Kar
ELTE - Eötvös Loránd Tudományegyetem, Budapest

2026. március 6.

1 Sor adattípus (Queue)

- Sor adattípus (Queue)
- Sor – Tömbös repr.
- Sor – Láncolt repr.
- Sor - Ciklikus láncolt repr.

2 Sor Feladatok

- "Dadogós" szavak
- Pascal-háromszög
- Legkisebb közös többszörös

Sor adattípus (Queue)

Sor adattípus (Queue)

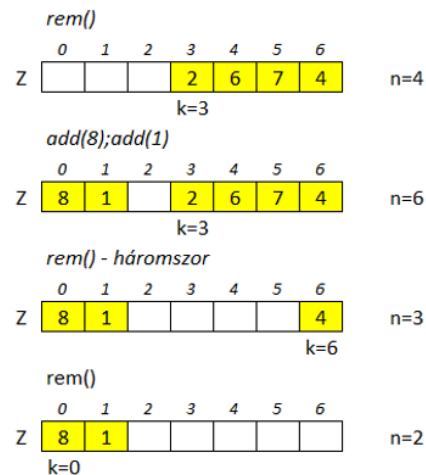
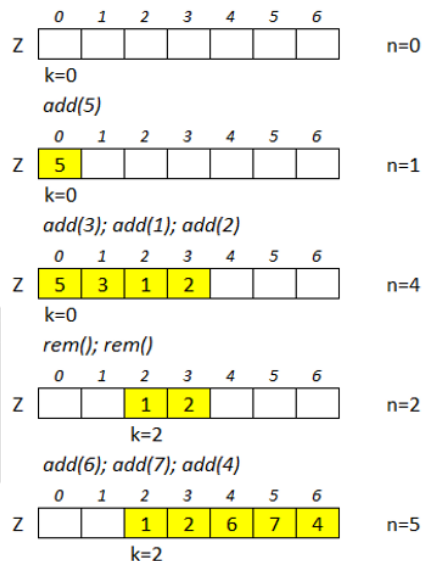
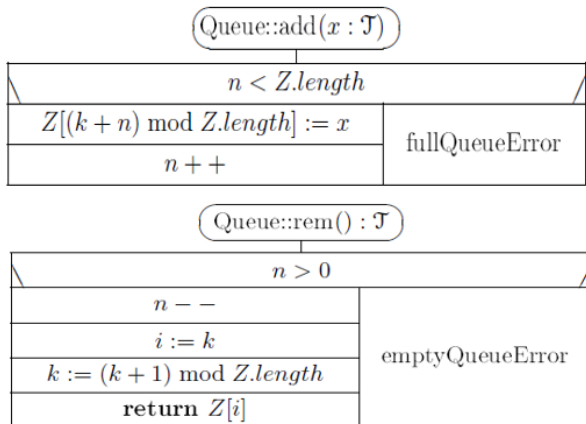
- A sor egy FIFO (First In First Out) adatszerkezet, azaz ellentétben a veremmel a legelőször behelyezett elemet vehetjük ki elsőként. Számos implementációja létezik a sornak, pl. statikus vagy dinamikus tömbbel (ld. előadás).
- Összesített időkomplexitás: $O(1)$

Queue
– $Z : \mathcal{T}[]$ // $\mathcal{T}$ is some known type ; $Z.length$ is the physical – constant $m0 : \mathbb{N}_+ := 16$ // length of the queue, its default is $m0$. – $n : \mathbb{N}$ // $n \in 0..Z.length$ is the actual length of the queue – $k : \mathbb{N}$ // $k \in 0..(Z.length - 1)$: the starting position of the queue in Z
+ Queue($m : \mathbb{N}_+ := m0$) { $Z := \text{new } \mathcal{T}[m]$; $n := 0$; $k := 0$ } // create an empty queue + add($x : \mathcal{T}$) // join x to the end of the queue + rem() : $\mathcal{T}$ // remove and return the first element of the queue + first() : $\mathcal{T}$ // return the first element of the queue + length() : $\mathbb{N}$ {return n} + isEmpty() : $\mathbb{B}$ {return $n = 0$} + ~ Queue() { delete Z } + setEmpty() { $n := 0$ } // reinitialize the queue

Sor adattípus (Queue)

- Sor egy tömbben:
 - A sor ciklikus, „körbe-körbe” jár.
 - A tömböt itt célszerű nullától indexelni.
 - Ha sor első elemének indexét (k), és a sor elemszámát (n) ismerjük, akkor ki is tudunk venni, és be is tudunk tenni konstans időben.
 - A sor első eleme: $Z[k]$ (ha nem üres)
 - Elemszám: n
 - Első szabad hely: $Z[k + n \bmod m]$

Sor tömbbel



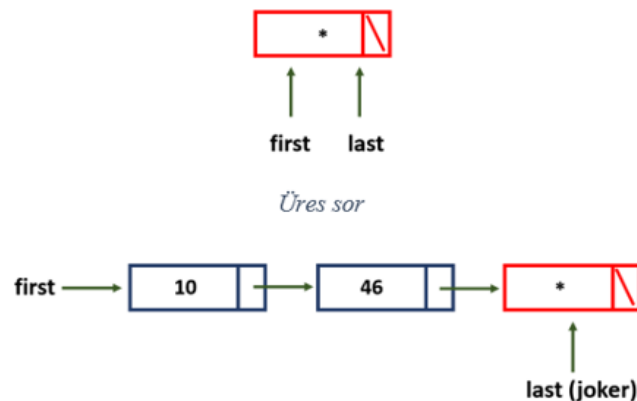
Sor – Láncolt reprezentáció

- Sor műveletek:
 - $\text{rem}() : T$
 - $\text{add}(x : T)$
 - $\text{first}() : T$
 - $\text{length}() : N$
 - $\text{isEmpty}() : B$
 - $\text{setEmpty}()$
- Műveletigény tömbös ábrázolás esetén: $\theta(1)$ (valamennyi művelet esetén)
- Ha az add műveletnél megengedjük, hogy a sort ábrázoló tömb mérete meg tudjon növekedni, annak műveletigénye mindig épp az aktuális hossz, de mivel ez ritkán fordul elő, átlagosan az add ekkor is konstans lépésszámúnak tekinthető.
- Ezt listás ábrázolás esetén is biztosítani kell!

Sor – Egyszerű láncolt reprezentáció

- Ötlet: Láncolt lista két pointerrel az első és az utolsó elemhez.
- Tartalmaz egy „joker” elemet a végén, amely a „következő ide beszúrandó elem” szerepét tölti be.
- Amikor új elemet adunk hozzá, azt a joker elem elé fűzzük be.
- Az „utolsó” pointer a joker elemre mutat.
- A joker elem egy fordított fejléc szerepét tölti be, így a lista sosem lesz üres.

Queue
-first, last: E1* // New pointers pointing to the first and last elements
-size: N
+ Queue() + add(x: T) // Adding new element at the end + rem(): T // Removing the first element + first(): T // Query the first element + length(): N + isEmpty(): B + ~Queue() + setEmpty()



Sor – Egyszerű láncolt struktogramjai

Queue::Queue()

first := last := new E1
first->next := 0
size := 0

A konstruktor létrehozza a joker elemet.

Queue::add(x: T)

last->next := new E1
last->key := x
last := last->next
last->next := 0
size := size + 1

Queue::setEmpty()

first ≠ last
p := first
first := first->next
delete p
size := 0

Queue::rem(): T

size = 0	
Error	x := first->key
	s := first
	first := first->next
	delete s
	size := size - 1
	return x

Queue::first(): T

size = 0	
Error	return first->key

Queue::isEmpty(): B

return size = 0

Queue::length(): N

return size

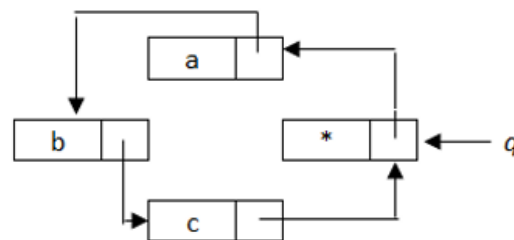
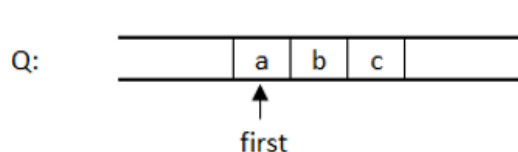
Queue::~~Queue()

first ≠ 0
p := first
first := first->next
delete p

- Műveletigény a setEmpty() és a destruktort kivéve : $\theta(1)$
(azok a lista hosszával arányosak)

Sor - Ciklikus láncolt reprezentáció

- Egy egyirányú, ciklikus lista ábrázolja a sort. Itt is van egy joker elem, ami a sor „végén” található. A joker elem mutat az első elemre.
- Egyetlen pointerre van szükség (az ábrán ez q), aminek a joker elemre kell mutatnia.
- A $\text{rem}()$ művelet esetén az első elem így azonnal elérhető a q segítségével és kifűzhető.
- Az $\text{add}()$ művelet esetén a joker elembe kerül sorba berakni kívánt érték, majd létrehozunk egy új joker elemet, befűzzük a láncba, és q -val erre az új elemre mutatunk.



Sor - Ciklikus láncolt struktogramjai

Queue::Queue()

```
q := new E1
q->next := q
size := 0
```

Queue::add(x: T)

```
r := q->next
q->key := x
q->next := new E1
q := q->next
q->next := r
size := size+1
```

Queue::setEmpty()

```
p := q->next
p ≠ q
  r := p
  p := p->next
  delete r
q->next := q
size := 0
```

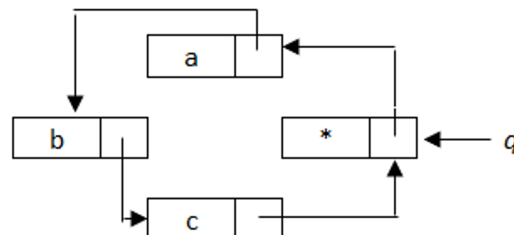
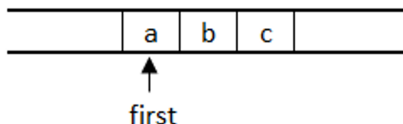
Queue::rem(): T

size = 0	
ERROR	r := q->next->next
	x := q->next->key
	delete q->next
	q->next := r
	size := size-1
	return x

Queue::~~Queue()

```
p := q->next
p ≠ q
  r := p
  p := p->next
  delete r
delete q
```

Q:



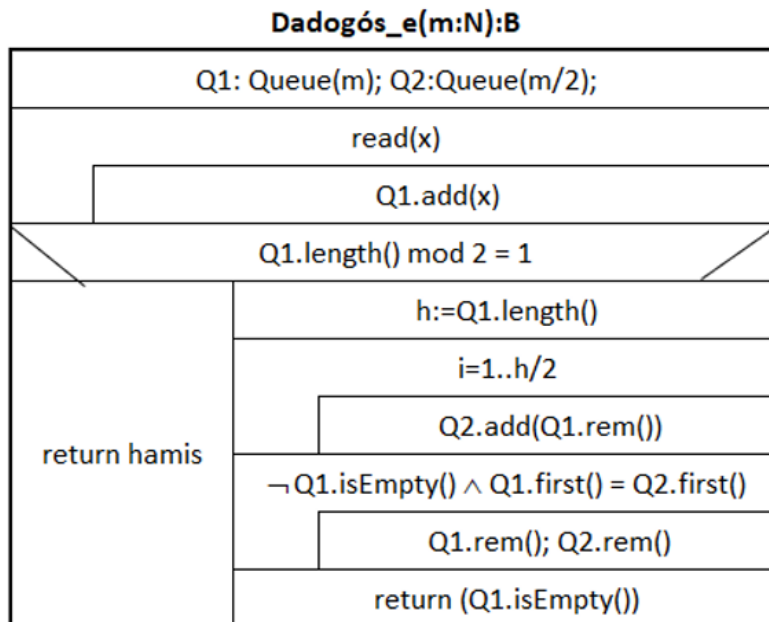
Sor Feldatok

"Dadogó" szavak problémája

- Bemenetről érkezik egy szöveg és el kell dönteni, hogy a bemeneti szöveg „dadogós-e” vagy sem.
- Példa: “aa”, “almaalma”, <> (üres szöveg is dadogósnak tekinthető)
- Ötlet:
 - Olvassuk be a szöveget egy sorba.
 - Ha a hossz páratlan, akkor nem „dadogós”.
 - Ha páros, az egyik felét másoljuk egy másik sorba, majd hasonlítsuk össze.

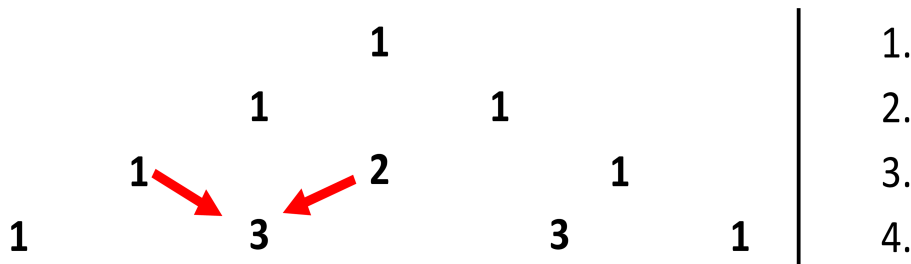
Megoldás: "Dadogó" szavak

- Inicializáljuk a sorokat.
- Beolvassuk a szöveget Q1-be.
- Ellenőrizzük, hogy páratlan-e.
- Q1 egyik felét átmásoljuk Q2-be.
- Összehasonlítjuk a sorokat:
 - Ha egyenlőek, folytatjuk.
 - Ha üres, visszatérünk az eredménnyel.

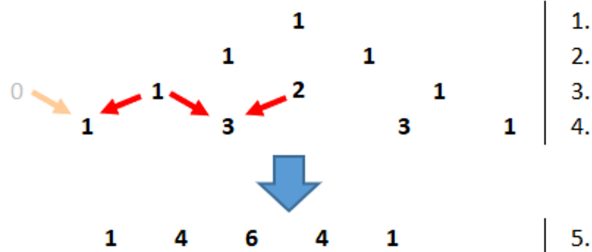


Pascal-háromszög generálása

- Sor használatával állítsuk elő a Pascal háromszög tetszőleges, k. sorát.
- Szorzást nem, csak összeadást használhat az algoritmus!
- (Forrás:
<https://www.mathsisfun.com/pascals-triangle.html>)



Pascal-háromszög generálása



Pascal(k: N ; Q: Queue)

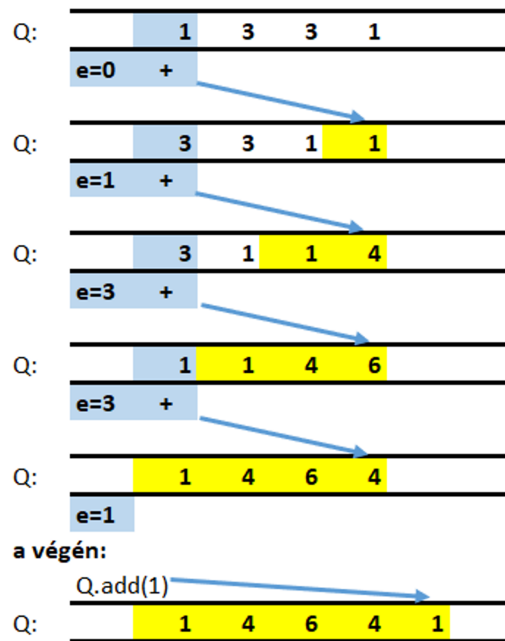
Q.setEmpty()
Q.add(1)
i := 2 to k
e := 0
j := 1 to i-1
Q.add(e + Q.first())
e := Q.rem()
Q.add(1)

i-1-edik sorból az i-edik előállítás:

(i-1)-szer ismételjük:

Q.add(e+Q.first())

e:=Q.rem()



Pascal-háromszög generálása

Pascal(k: N; Q:Queue)

Sor, verem (nagyobb objektum) mindig referencia szerint adódik át, nem kell jelölni!

Q.setEmpty()
Q.add(1)
i := 2 to k
e := 0
j := 1 to i-1
Q.add(e + Q.first())
e := Q.rem()
Q.add(1)

Betesszük az első sorban található 1-est, így a háromszög első sora Q-ban van.

A háromszög i. sora az i-1. sor segítségével számítható ki. Amikor a ciklusba belépünk Q-ban az i-1. sor elemei vannak.

A Q sorból kiszedegetjük az elemeket, közben már állítjuk elő a háromszög i-dik sorát.

Kihasználjuk, hogy az i-1. sor pontosan i-1 elemből áll,

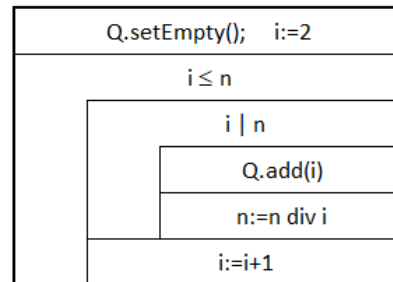
e-ben mindig az előző menetben kivett elem van, kezdetben pedig nulla.

Még egy 1-est beteszünk Q végére, ezzel a háromszög i. sora Q-ban előállt.

Két rendezett sor összefésülése

- Q1 és Q2 egy-egy 1-nél nagyobb természetes szám prímtényezős felbontását tartalmazza.
- A sorok növekvően rendezettek.
- Q2-ben állítsuk elő a két szám **legkisebb közös többszörösének** prímtényezős felbontását.
- Q1 sor tartalmát őrizzük meg!
- Példa bemenet:
 - $Q1 = \langle 2, 2, 5, 11 \rangle$
 - $Q2 = \langle 2, 3, 5, 5, 7 \rangle$
- Eredmény:
 - $Q1 = \langle 2, 2, 5, 11 \rangle$
 - $Q2 = \langle 2, 2, 3, 5, 5, 7, 11 \rangle$

prímtényezősFelbontás($n:N, Q:Queue$) Ef: $n > 1$



Két rendezett sor összefésülése

LKKT(Q1: Queue, Q2: Queue)

Q1.add(-1); Q2.add(-1)			// -1 végjel berakása a sorokba
Q1.first() $\neq$ -1 $\vee$ Q2.first() $\neq$ -1			
$Q2.first() = -1 \vee (Q1.first() \neq -1 \wedge Q1.first() < Q2.first())$	$Q1.first() \neq -1 \wedge Q2.first() \neq -1 \wedge Q1.first() = Q2.first()$	$Q1.first() = -1 \vee (Q2.first() \neq -1 \wedge Q1.first() > Q2.first())$	
	Q2.add(Q1.first())	Q2.add(Q2.rem())	
	Q1.add(Q1.rem())		
Q1.rem(); Q2.rem()			// -1 végjel eltüntetése a sorokból

	Q1	Q2
=	2,2,5,11,-1	2,3,5,5,7,-1
<	2,5,11,-1,2	3,5,5,7,-1,2
>	5,11,-1,2,2	3,5,5,7,-1,2,2
=	5,11,-1,2,2	5,5,7,-1,2,2,3
>	11,-1,2,2,5	5,7,-1,2,2,3,5
>	11,-1,2,2,5	7,-1,2,2,3,5,5
Q2.first()=-1	11,-1,2,2,5	-1,2,2,3,5,5,7
	-1,2,2,5,11	-1,2,2,3,5,5,7,11

Köszönöm a figyelmet!